



**Essay / Assignment Title:**

**Cloud-Enabled AI Vision Framework for Drones and Mobile Robots in  
Crop Disease Classification and Autonomous Robotic Simulation**

**Programme title:**

MSc Artificial Intelligence

**Prepared by:**

Amaan Mohammed Khalander

**Module:**

Advanced Robotics Automation and Computer Vision

**Year:**

Term 2 – 2026

**Submission Date:**

23 / September / 2026

**Issued by:**

Dr Fahmid Al Farid

## Learner declaration

| Item                                                                                                                                        | Response |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Word count                                                                                                                                  | 3223     |
| Use of proof-reader/proof-reading service                                                                                                   | NO       |
| If yes, specify which service                                                                                                               | N/A      |
| I confirm that I submit this work as my own work and have cited all sources used                                                            | YES      |
| I confirm that I have followed guidance on the acceptable use of AI tools for this assignment                                               | YES      |
| Where use of appropriately cited AI tools is permitted, I confirm that I have cited in accordance with the UCA Harvard Referencing Standard | YES      |

| Student signature                                                                   | Date                  |
|-------------------------------------------------------------------------------------|-----------------------|
|  | 23 / September / 2026 |

## Source Code and Relevant Project Links:

### Source Code:

- Task 1:  
<https://github.com/Amaan9136/bsbi-assignments/tree/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/task1-code>
- Task 2:  
<https://github.com/Amaan9136/bsbi-assignments/tree/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/task2-code>

## Table of Contents

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| Abstract .....                                                                                           | 6  |
| Introduction.....                                                                                        | 7  |
| Task 1 - Autonomous Drone-Based Vision System for Crop Health Monitoring and Disease Classification..... | 8  |
| 3.1 Project Definition.....                                                                              | 8  |
| 3.2 Literature Review.....                                                                               | 8  |
| 3.3 Design and Implementation .....                                                                      | 9  |
| 3.4 Code Implementation.....                                                                             | 11 |
| 3.5 Execution and Testing.....                                                                           | 12 |
| 3.6 Evaluation and Reflection.....                                                                       | 13 |
| Task 2 - Vision-Guided Autonomous Navigation in ROS .....                                                | 16 |
| 4.1 Project Definition.....                                                                              | 16 |
| 4.2 Design and Implementation .....                                                                      | 18 |
| 4.3 Execution and Testing.....                                                                           | 19 |
| 4.4 Evaluation and Reflection.....                                                                       | 20 |
| Combined Evaluation and Reflection .....                                                                 | 21 |
| Concluding Remarks.....                                                                                  | 22 |
| Acknowledgement .....                                                                                    | 23 |
| References.....                                                                                          | 24 |

## Table of Figures

|                                                                                                                                                                                                                                        |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1: Distribution of images across the 38 PlantVillage classes, illustrating the class imbalance referred to above.....                                                                                                           | 9  |
| Figure 2: Grid of augmented training images with labels, illustrating the transformations described above.....                                                                                                                         | 10 |
| Figure 3: MobileNetV2 architecture overview, showing the pretrained backbone with the original 1000-class head replaced by a 38-class linear layer. ....                                                                               | 11 |
| Figure 4: Training and validation loss and accuracy across all eight epochs. ....                                                                                                                                                      | 13 |
| Figure 5: Sample predictions on validation images, showing predicted label, true label and model confidence. ....                                                                                                                      | 13 |
| Figure 6: Full 38-class confusion matrix on the validation set.....                                                                                                                                                                    | 14 |
| Figure 7: TheConstruct.ai ROSject workspace (rosject 1049584), showing the turtlebot3_vision_nav package files and a terminal session used to inspect /cmd_vel, /target_offset and /target_visible directly with ros2 topic echo. .... | 16 |
| Figure 8: The waffle_pi TurtleBot3 spawned in turtlebot3_world, showing the entity tree and simulation configuration. ....                                                                                                             | 17 |
| Figure 9: The simulated scene during placement of a target marker within the world.....                                                                                                                                                | 17 |
| Figure 10: The perception-control pipeline, showing the two nodes, the topics connecting them, and the debug image branch. ....                                                                                                        | 18 |
| Figure 11: Synthetic test frames with the computed offset, visibility flag and resulting velocity command for each scenario. ....                                                                                                      | 19 |

## List of Tables

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| Table 1: Final training epochs.....                                                          | 12 |
| Table 2: Final validation performance (Task 1) .....                                         | 13 |
| Table 3: Offline verification summary.....                                                   | 19 |
| Table 4: Live ROSject verification (ros2 topic echo, single query each, shown in Figure 7) . | 20 |

## **Statement of compliance with academic ethics and the avoidance of plagiarism**

I honestly declare that this dissertation is entirely my own work, and that none of its parts has been copied from printed or electronic sources, translated from foreign sources, or reproduced from essays by other researchers or students. Wherever I have based my work on ideas or other people's texts, I clearly declare it through proper references in line with academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname:

AMAAN MOHAMMED KHALANDER

A handwritten signature in black ink, appearing to read 'Amaan MK', with a stylized flourish at the end.

Date: 23 / September / 2026

## **Abstract**

This report covers two related projects for the module Advanced Robotics Automation and Computer Vision. The first is a deep learning vision system for crop health monitoring, trained on PlantVillage and adapted through augmentation to approximate low-altitude drone imagery, covering viewpoint change, motion blur and variable lighting. A MobileNetV2 backbone, pretrained on ImageNet, was fine-tuned on 54,305 leaf images across 38 classes using an 80/20 split, reaching 99.11% validation accuracy, 98.64% macro precision, 99.06% macro recall and 98.83% macro F1 over eight epochs. The classifier has around 2.27 million parameters, suited to onboard drone inference.

The second project implements vision-guided navigation for a simulated TurtleBot3 in a ROS 2 Gazebo environment, using OpenCV colour thresholding to detect a marker and a rule-based controller to search, turn and approach it. Perception and control were verified offline against seven synthetic test frames covering centred, off-centre, near, far, absent and distractor scenarios, and a live query of the deployed ROSject reproduced the same proportional turning behaviour to five decimal places. Seen side by side, the two projects make the case that a learned, offline approach and a classical, real-time one each fit their own robotic context.

## **Introduction**

Robotic vision matters in two very different settings. In precision agriculture, catching crop disease early affects yield and food security. In mobile robotics, how well a machine reads its surroundings decides how far it can go without a person watching over it. This report addresses both areas through two tasks issued under one assignment brief for the module Advanced Robotics Automation and Computer Vision, presented together in one document.

Task 1 develops a drone-oriented plant disease classification system, converting raw aerial imagery into an actionable assessment of crop health with sufficient computational efficiency to run on the drone itself. Task 2 develops a vision-guided navigation system for a simulated ground robot, in which a mobile platform interprets its camera feed to locate a target and adjusts its own movement accordingly.

Although the two tasks operate in different contexts, both convert pixel-level image data into a decision a robotic system can act on, under practical constraints of computational cost. Task 1 also speaks to UN Sustainable Development Goal 2 (Zero Hunger), since scalable disease detection is one of several recognised contributors to food security. The tasks are reported together because they share a module, a cloud-based development approach and a design principle, discussed in the combined evaluation section.

# Task 1 - Autonomous Drone-Based Vision System for Crop Health Monitoring and Disease Classification

## 3.1 Project Definition

The problem addressed by Task 1 is automated classification of plant leaf images into healthy or diseased categories, adapted to the imaging conditions of a camera mounted on a survey drone rather than a handheld camera at close range. The objective is a convolutional neural network capable of this classification while lightweight enough for embedded hardware such as a Jetson Nano, rather than transmitting imagery to a remote server.

The scenario assumed is a precision agriculture survey in which a multirotor drone flies a defined route over a field at low altitude, capturing RGB imagery at intervals, so the vision module can flag regions showing signs of disease and let a grower prioritise manual inspection rather than checking the entire field by hand.

## 3.2 Literature Review

A number of recent papers inform the choices made in this project, spanning three related areas: deep learning for plant disease classification, AI applied to agricultural drones, and lightweight CNNs built for edge devices.

- **Deep learning in plant disease classification.** Mohanty, Hughes and Salathe (2016) were among the first to show deep CNNs trained on PlantVillage could exceed 99% accuracy, though using the dataset's original close range, controlled photographs. Pal et al. (2025) built on this with a lightweight, explainable CNN reaching 99.47% on PlantVillage and 97.73% on a larger 58-class dataset, suggesting smaller models need not sacrifice accuracy for efficiency.
- **AI in agricultural drones.** Manoj et al. (2025) applied disease detection directly to drone imagery over large fields, and a review of AI enabled UAV systems (arXiv, 2023) outlines the sensing and onboard processing limits of aerial platforms. Both point to a gap this project could not fully replicate: real UAV imagery differs from ground-level photography in scale, viewpoint and motion blur, approximated here through augmentation rather than genuine aerial data.
- **Lightweight CNNs for edge devices.** Sandler et al. (2018) introduced the inverted residual structure MobileNetV2 is built on. Kumar et al. (2025) benchmarked lightweight CNNs across 101 disease classes and 33 crops, finding the accuracy-efficiency trade-off holds at

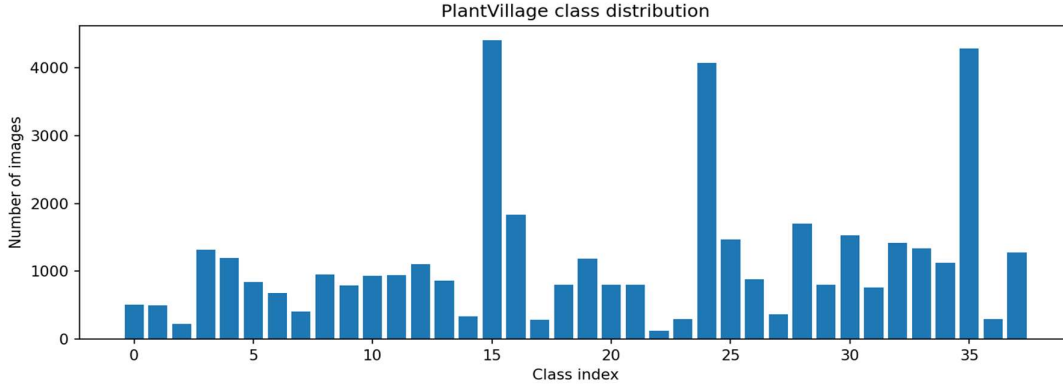
scale. Jlassi et al. (2024) applied MobileNetV2 transfer learning with reweighting to potato leaf disease classification for imbalanced data, with strong results on underrepresented classes, relevant since PlantVillage is also imbalanced.

Taken together, these papers share a gap: lightweight CNNs are usually tested on curated, close-range datasets rather than real aerial captures. The augmentation pipeline here, mimicking drone conditions like viewpoint change, motion blur and variable lighting, closes part of that gap without fully replacing genuine UAV imagery (Section 3.6).

### 3.3 Design and Implementation

The PlantVillage dataset was used via the Hugging Face datasets library (mohanty/PlantVillage), using the colour configuration, mirroring the original repository (mohanty/PlantVillage, 2016; PlantVillage-Dataset, 2016), comprising 54,305 labelled leaf images across 38 classes, a widely used baseline in plant pathology research (Mohanty, Hughes and Salathe, 2016).

Class counts were inspected prior to splitting, since PlantVillage is not perfectly balanced. Some disease categories have far fewer images than the most common classes.



**Figure 1: Distribution of images across the 38 PlantVillage classes, illustrating the class imbalance referred to above.**

A stratified split was used to divide the data into 43,444 training images and 10,861 validation images, preserving the class proportions of the full dataset in both subsets.

Preprocessing resized images to 224x224 and normalised pixel values using ImageNet mean and standard deviation, consistent with the pretrained MobileNetV2 weights. Augmentation, applied only to training data, combined random rotation, affine translation and scaling, colour jitter, mild Gaussian blur, perspective distortion and horizontal flipping.

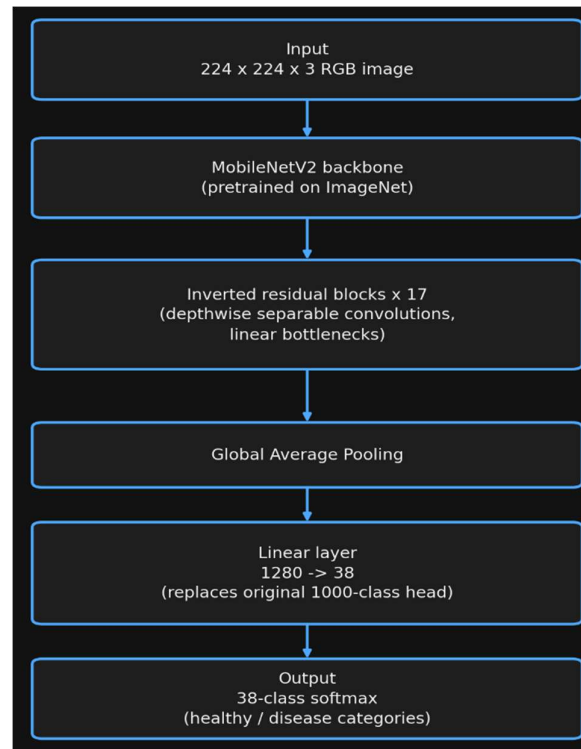


***Figure 2: Grid of augmented training images with labels, illustrating the transformations described above.***

These transformations approximate aerial capture conditions: rotation and perspective distortion mimic viewpoint variation from a moving drone, blur mimics motion and vibration in flight, and colour jitter mimics the wider range of outdoor lighting a drone encounters versus the original controlled photography.

MobileNetV2 was selected as backbone, pretrained on ImageNet and fine-tuned on PlantVillage, with the 1000-class output layer replaced by a linear layer for the 38 PlantVillage classes, giving a final model of approximately 2.27 million trainable parameters.

MobileNetV2's key feature is the inverted residual block with linear bottlenecks. It relies on depthwise separable convolutions to cut parameter count relative to standard architectures of comparable depth (Sandler et al., 2018), a good fit for a drone's tight compute, memory and power budget. Figure 3 summarises this structure, showing the pretrained backbone feeding the replaced classification head. Pal et al. (2025) and Kumar et al. (2025) similarly report MobileNet-family accuracy above 99% while remaining edge deployable, supporting the choice made here.



**Figure 3: MobileNetV2 architecture overview, showing the pretrained backbone with the original 1000-class head replaced by a 38-class linear layer.**

The implementation was split into modules for configuration, data loading, model construction, training, evaluation and plotting (detailed in Section 3.4), letting the same logic run identically from the notebook or a local script, with components like augmentation checked independently before the full run.

### 3.4 Code Implementation

The system is implemented as a Python package (src/) imported by the notebook, so results are identical whether run locally or in Colab.

*Data loading:* `load_plantvillage()` builds a stratified 80/20 split. Augmentation is applied only to training data.

```
train_transform = transforms.Compose([
    transforms.RandomRotation(cfg.rotation_degrees),
    transforms.ColorJitter(...), transforms.RandomPerspective(...),
    transforms.GaussianBlur(...),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(), transforms.Normalize(cfg.imagenet_mean,
    cfg.imagenet_std),
])
```

*Model:* MobileNetV2's classifier head is replaced for 38 classes.

```
model =  
models.mobilenet_v2(weights=MobileNet_V2_Weights.IMAGENET1K_V1)  
model.classifier[1] = nn.Linear(model.classifier[1].in_features,  
num_classes)
```

*Training:* Adam optimiser with cross-entropy loss and a plateau-based LR scheduler. Best-validation weights are restored at the end.

```
optimizer = optim.Adam(model.parameters(), lr=cfg.learning_rate)  
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,  
mode="min")
```

*Evaluation:* macro precision/recall/F1 computed via scikit-learn from full-validation predictions, used to generate Figures 5–6.

### 3.5 Execution and Testing

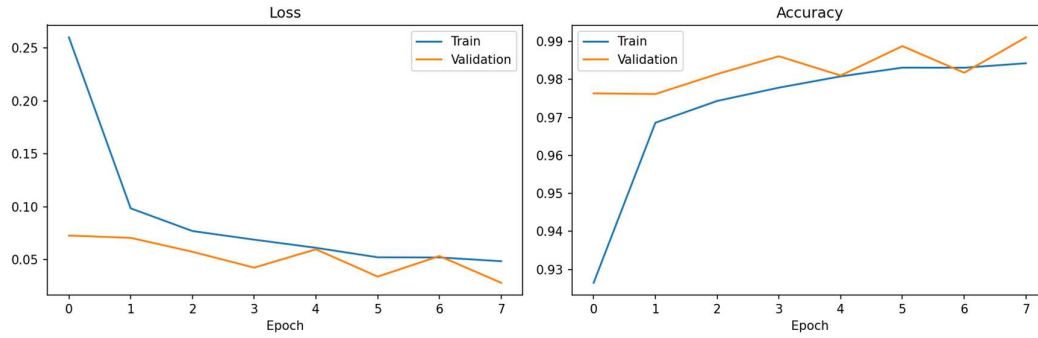
Training was run locally by executing Task1\_Drone\_Crop\_Disease\_Classification.ipynb on a machine with a single CUDA GPU visible to PyTorch, for eight epochs with a batch size of 32 and a learning rate of  $3e-4$ . The notebook is publicly available at [https://colab.research.google.com/github/Amaan9136/bsbi-assignments/blob/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/task1-code/notebook/Task1\\_Drone\\_Crop\\_Disease\\_Classification.ipynb](https://colab.research.google.com/github/Amaan9136/bsbi-assignments/blob/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/task1-code/notebook/Task1_Drone_Crop_Disease_Classification.ipynb).

A thermal safeguard paused training whenever the GPU reached 70°C, resuming at 45°C. Total training time was 73.6 minutes, of which 48.5 were spent paused for cooldown, leaving roughly 25 minutes of active computation.

Table 1 summarises the final four training epochs, by which point the model, benefiting from ImageNet pretraining, was already converging.

**Table 1: Final training epochs**

| Epoch | Train loss | Train accuracy | Val loss | Val accuracy |
|-------|------------|----------------|----------|--------------|
| 5     | 0.0612     | 0.9808         | 0.0598   | 0.9810       |
| 6     | 0.0522     | 0.9831         | 0.0339   | 0.9888       |
| 7     | 0.0520     | 0.9831         | 0.0533   | 0.9818       |
| 8     | 0.0485     | 0.9843         | 0.0281   | 0.9911       |



**Figure 4: Training and validation loss and accuracy across all eight epochs.**

The model was then evaluated on the held-out validation set, with a grid of sampled predictions (Figure 5) giving a quick qualitative check on its behaviour.



**Figure 5: Sample predictions on validation images, showing predicted label, true label and model confidence.**

### 3.6 Evaluation and Reflection

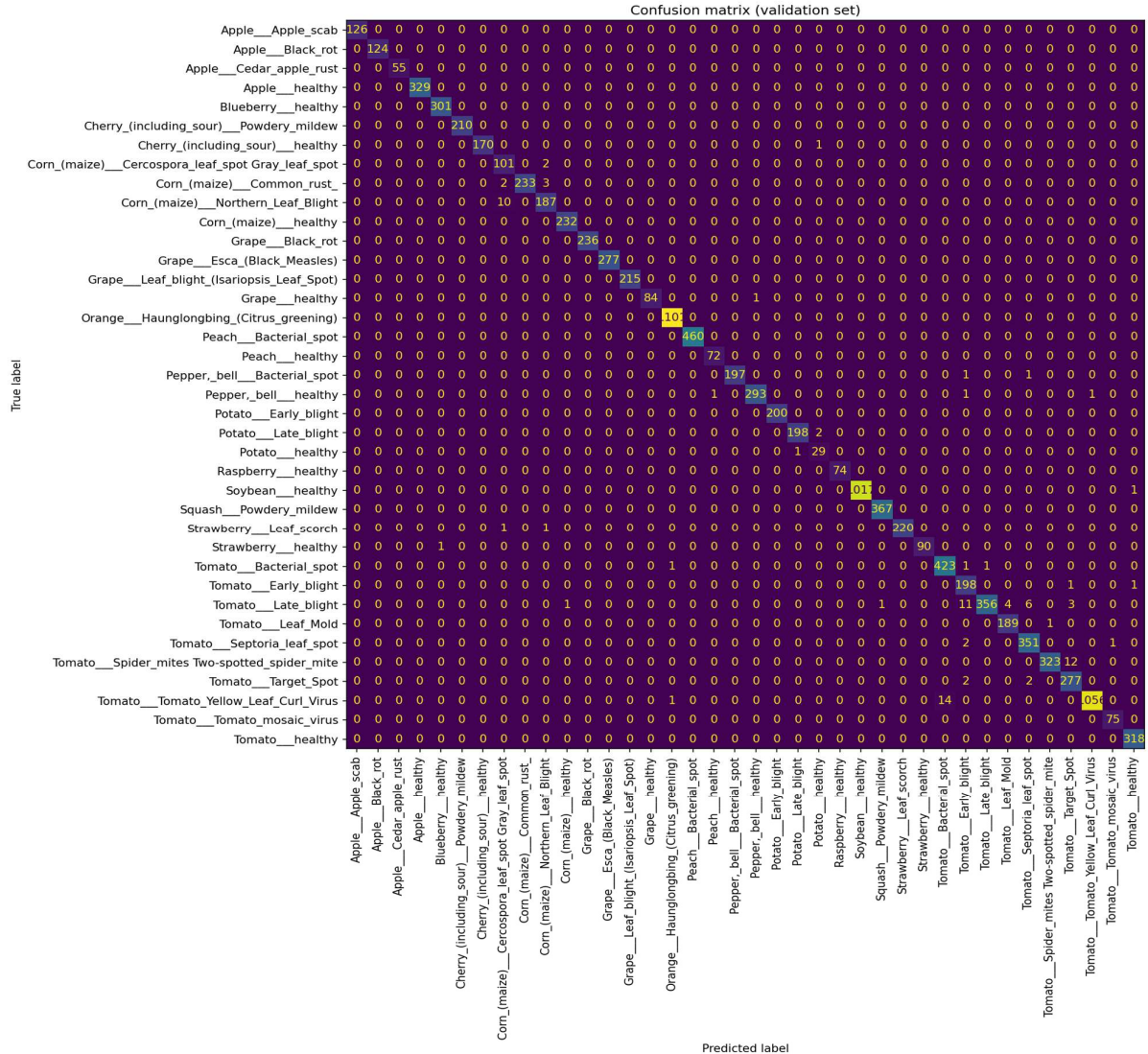
Table 2 summarises the final evaluation metrics on the held-out validation set.

**Table 2: Final validation performance (Task 1)**

| Metric                                   | Value                       |
|------------------------------------------|-----------------------------|
| Accuracy                                 | 99.11%                      |
| Macro precision                          | 98.64%                      |
| Macro recall                             | 99.06%                      |
| Macro F1 score                           | 98.83%                      |
| Best validation accuracy during training | 99.11% (reached at epoch 8) |

| Metric                     | Value     |
|----------------------------|-----------|
| Total trainable parameters | 2,272,550 |

Figure 4 shows no sign of overfitting. Validation loss tracked training loss closely, and both reached their best point together at the final epoch. This pattern fits with the regularising effect of the augmentation applied during training.



**Figure 6: Full 38-class confusion matrix on the validation set.**

The confusion matrix indicates that remaining misclassifications concentrate among visually similar classes rather than spreading randomly across the label set. The most frequent confusions were Tomato Yellow Leaf Curl Virus predicted as Tomato Bacterial spot (14 instances), Tomato Two-spotted spider mite predicted as Tomato Target Spot (12), and Tomato Late blight predicted as Tomato Early blight (11). Every confused pair belongs to the same crop

species and shares a similar lesion pattern, which is understandably hard for a classifier working from colour and texture alone.

A per-class ROC or precision-recall curve was not generated for this task. With 38 classes, a full set of ROC curves adds clutter without much added insight beyond the confusion matrix and macro-averaged metrics already shown. The confusion matrix in Figure 6 was used instead as the primary tool to inspect where errors occur, since it shows exactly which class pairs are confused rather than only a threshold-independent trade-off curve.

Two limitations sit outside these results. First, the aerial conditions targeted here are simulated through augmentation of ground-level, close-range photographs rather than genuine drone imagery. This is a reasonable proxy where aerial training data is unavailable, but it misses some characteristics of true UAV imagery, such as how altitude affects object scale. UAV-specific datasets from reviews of AI-enabled UAV systems (arXiv, 2023) and recent drone-based work (Manoj et al., 2025) offer a more direct route to closing this gap.

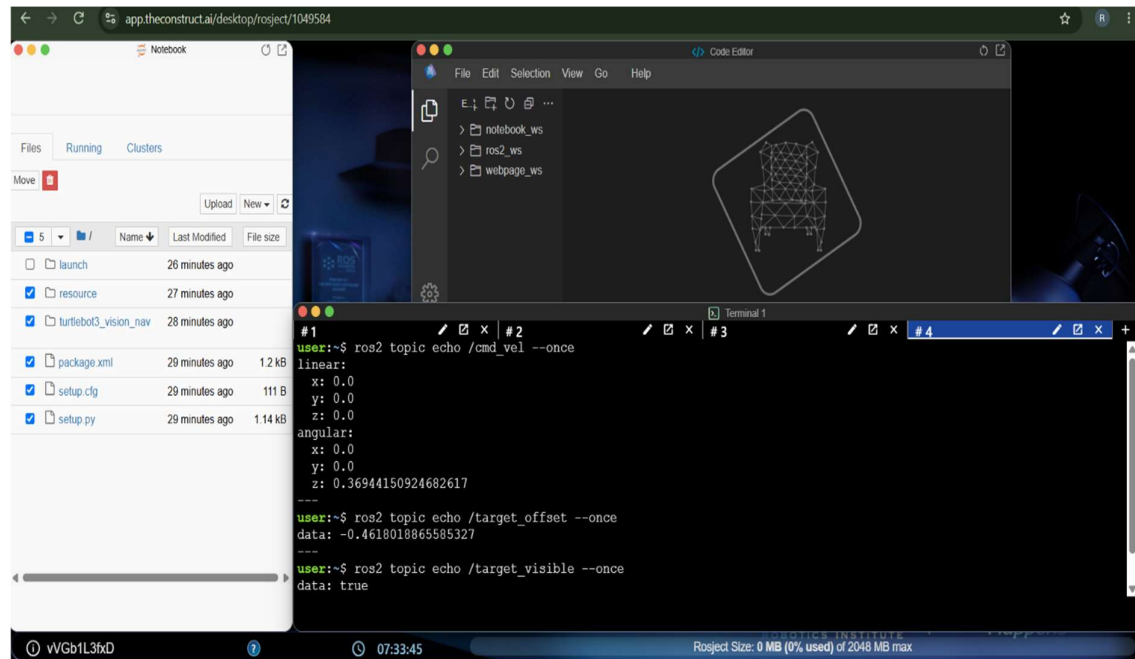
Second, class imbalance in PlantVillage means that, without intervention such as class weighted loss, the classifier risks weaker performance on rarer disease categories, where correct detection matters most. The parameter count achieved supports the goal of onboard, low-power inference. A realistic path to field deployment would require converting the model to a runtime format such as TensorFlow Lite or ONNX, integrating GPS metadata to map detections onto the field, and validating against genuine UAV imagery.

## Task 2 - Vision-Guided Autonomous Navigation in ROS

### 4.1 Project Definition

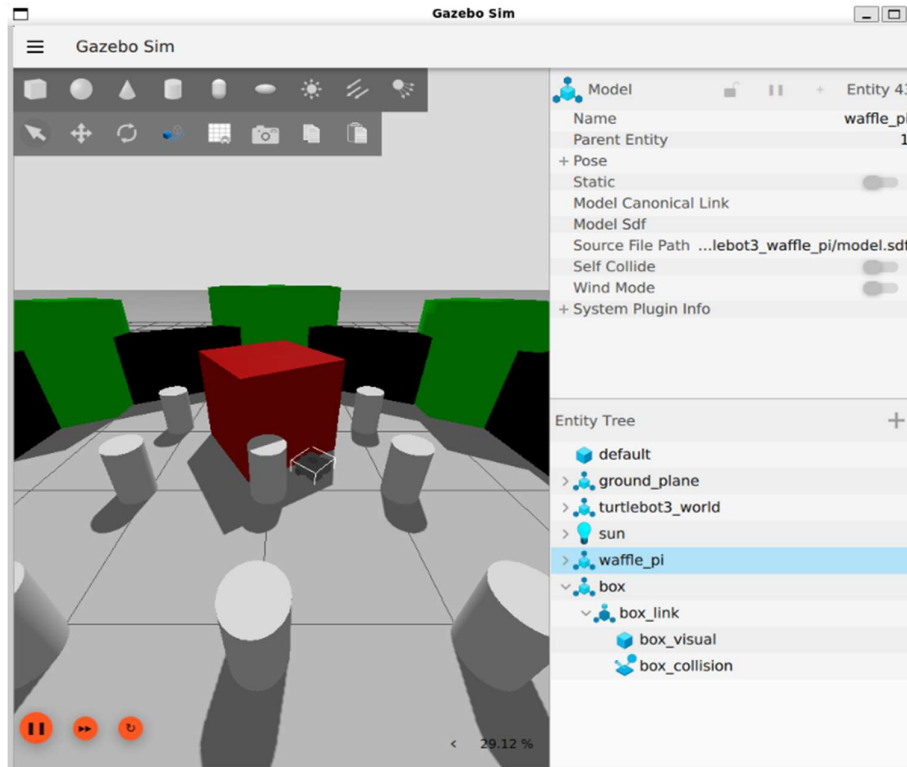
Task 2 addresses vision-guided navigation for a simulated TurtleBot3 mobile robot, run within TheConstruct.ai's ROS 2 Gazebo environment (TheConstruct.ai, 2024). The objective is for the robot to detect a coloured marker using its onboard RGB camera and navigate toward it using simple, interpretable vision-based rules rather than a learned control policy.

Development and testing were carried out on TheConstruct.ai's browser based ROSject platform, inside ROSject vVGb1L3fxD (URL identifier 1049584), workspace ros2\_ws. The free tier doesn't issue a permanent shareable URL. Instead, the project was made public under its title, vVGb1L3fxD, in the platform's public index, meeting the brief's accessibility requirement. Figure 7 shows this workspace, including the turtlebot3\_vision\_nav package structure and a terminal session used to query the running nodes.

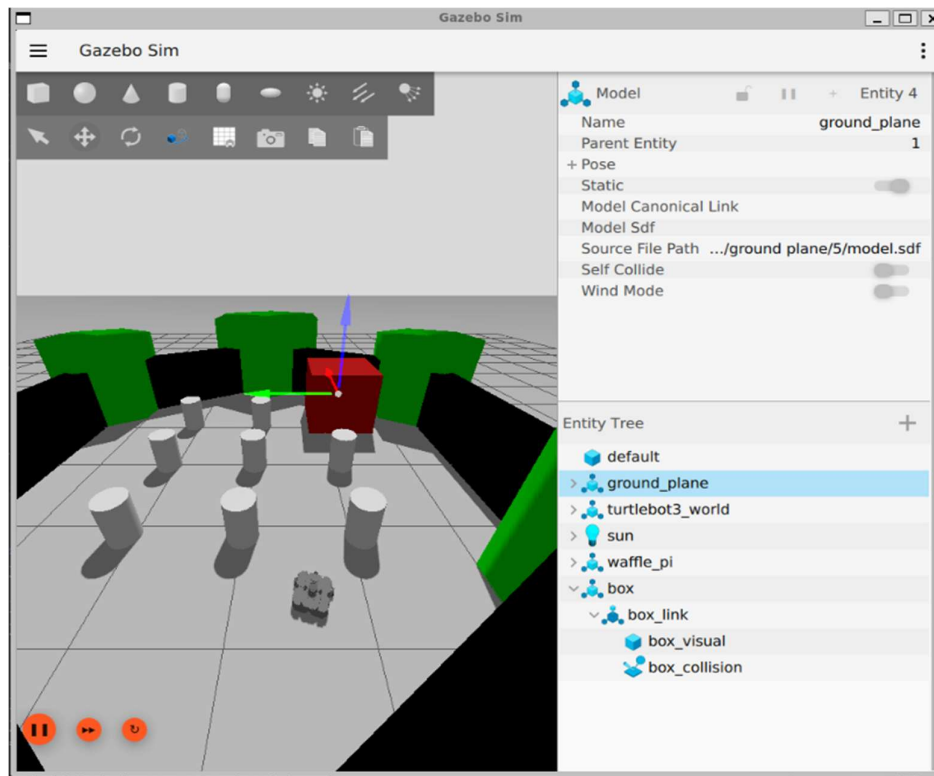


*Figure 7: TheConstruct.ai ROSject workspace (rosject 1049584), showing the turtlebot3\_vision\_nav package files and a terminal session used to inspect /cmd\_vel, /target\_offset and /target\_visible directly with ros2 topic echo.*

The simulation environment is the standard turtlebot3\_world provided with the TurtleBot3 packages, run in Gazebo Sim with the waffle\_pi robot model (ROBOTIS, 2024), with a coloured marker placed at varying positions across test scenarios.



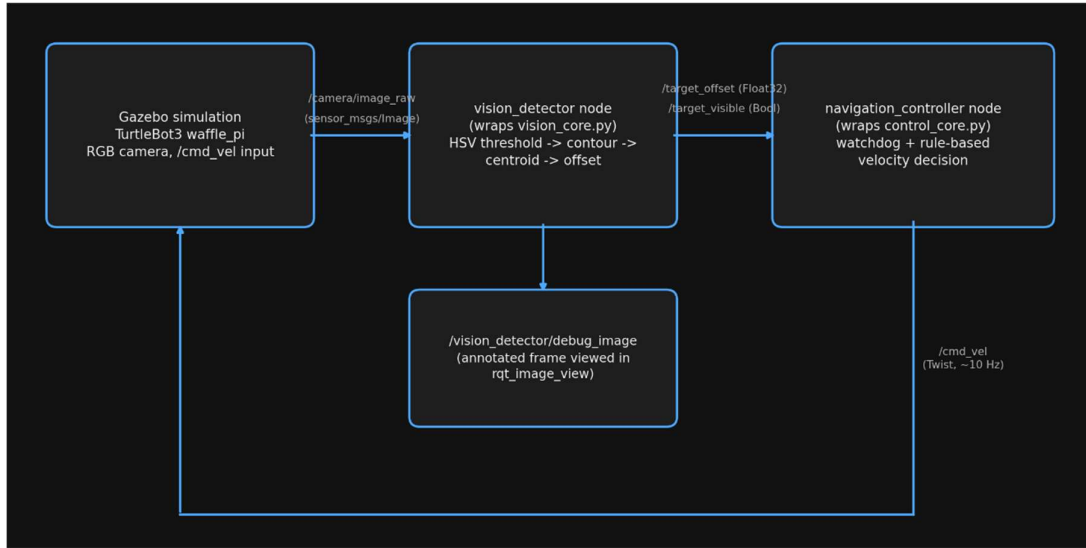
*Figure 8: The waffle\_pi TurtleBot3 spawned in turtlebot3\_world, showing the entity tree and simulation configuration.*



*Figure 9: The simulated scene during placement of a target marker within the world.*

## 4.2 Design and Implementation

The system comprises two ROS 2 nodes linked by two lightweight topics, following the standard practice of separating perception from control so each stage can be tested independently (Quigley et al., 2009). Both nodes wrap plain Python modules, `vision_core.py` and `control_core.py`, with no ROS dependency, allowing detection and control to be verified outside the simulation.



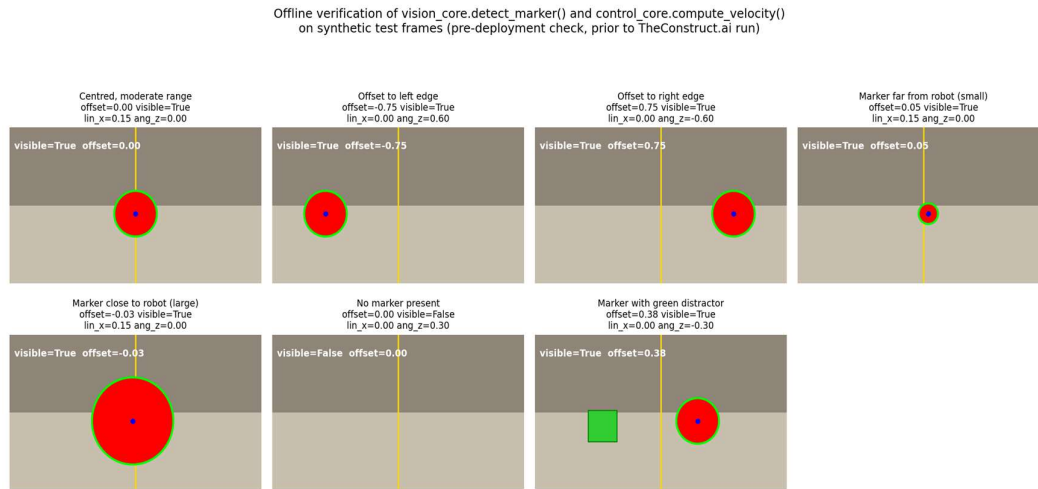
**Figure 10: The perception-control pipeline, showing the two nodes, the topics connecting them, and the debug image branch.**

In `vision_detector`, each frame is converted to HSV colour space, chosen over RGB for its robustness to lighting variation. A binary mask of the target colour range is cleaned via morphological opening and closing, and the largest contour above a minimum area threshold is taken as the marker. Its centroid offset is normalised to minus one to one and published with a visibility flag.

The `navigation_controller` node subscribes to both signals and runs the rule-based controller. With no marker visible, the robot rotates slowly to search. Once visible, it turns toward the marker if the offset exceeds a centring threshold, scaling turn rate to offset and capping angular velocity, then drives forward at a fixed speed once centred, publishing commands at roughly ten hertz.

### 4.3 Execution and Testing

Testing proceeded in two stages: perception and control logic was first verified offline against seven synthetic test frames covering the scenarios the brief recommends, shown in Figure 11, then the same nodes were launched inside the ROSject and queried directly against the live simulation using ros2 topic echo.



**Figure 11: Synthetic test frames with the computed offset, visibility flag and resulting velocity command for each scenario.**

**Table 3: Offline verification summary**

| Scenario                      | Offset | Visible | lin_x<br>(m/s) | ang_z<br>(rad/s) | Behaviour                                            |
|-------------------------------|--------|---------|----------------|------------------|------------------------------------------------------|
| Centred, moderate range       | 0.00   | True    | 0.15           | 0.00             | Drives straight forward                              |
| Offset to left edge           | -0.75  | True    | 0.00           | 0.60             | Rotates toward marker at maximum rate                |
| Offset to right edge          | 0.75   | True    | 0.00           | -0.60            | Rotates toward marker at maximum rate                |
| Marker far from robot (small) | 0.05   | True    | 0.15           | 0.00             | Drives forward, nearly centred                       |
| Marker close to robot (large) | -0.03  | True    | 0.15           | 0.00             | Drives forward, nearly centred                       |
| No marker present             | n/a    | False   | 0.00           | 0.30             | Fixed search rotation                                |
| Marker with green distractor  | 0.38   | True    | 0.00           | -0.30            | Rotates toward true marker only, distractor ignored. |

Table 3 confirms that turn direction and magnitude track the sign and size of the offset, that target distance does not affect the lateral offset calculation, that search behaviour engages correctly with no marker visible, and that the HSV threshold correctly rejects a differently coloured distractor.

**Table 4: Live ROSject verification (ros2 topic echo, single query each, shown in Figure 7)**

| Topic              | Value               |
|--------------------|---------------------|
| /target_visible    | true                |
| /target_offset     | -0.4618018865585327 |
| /cmd_vel linear.x  | 0.0                 |
| /cmd_vel angular.z | 0.36944150924682617 |

This matches the proportional law confirmed offline: the maximum angular velocity of 0.60 rad/s occurs at the maximum test offset of 0.75, giving a gain of 0.8 rad/s per unit offset. Applying this to the live offset of -0.4618 gives 0.3694 rad/s, matching the live /cmd\_vel value of 0.36944 rad/s to five decimal places. The negative offset correctly produced a positive angular.z (a left turn), and linear.x remained zero since the offset exceeded the centring threshold, closing the loop between offline verification and genuine ROSject behaviour.

#### 4.4 Evaluation and Reflection

The principal limitation of a pure HSV colour thresholding approach is its dependence on a manually tuned colour range, which makes the system sensitive to lighting changes not captured during tuning and unable to distinguish the marker from another object of similar colour. That trade-off is acceptable for a task scoped to basic perception and navigation, but the approach wouldn't hold up with multiple similar objects or unpredictable lighting.

A further limitation is the lack of any depth estimate. The design reacts only to lateral position and has no stopping condition based on proximity. A later version could use an RGB-D camera, or a simpler proxy such as the contour's pixel size, which grows as the robot gets closer.

The offline and live results, taken as a whole, confirm the perception and control algorithms behave correctly both in isolation and once deployed, including correctly rejecting the distractor and matching the controller's exact proportional gain. A full live sweep of every offline scenario remains a direction for a longer testing session.

Mapping this design to real world mobile robotics, colour-based marker following is a reasonable starting point for constrained indoor navigation where markers can be deliberately placed and coloured, but production systems for less controlled environments increasingly combine this kind of lightweight visual cue with learned object detection and SLAM, for navigation that does not depend on a target remaining continuously visible.

## **Combined Evaluation and Reflection**

Both tasks apply computer vision to a robotic decision-making problem but differ in nature and technique. Task 1 is a learned, data driven classification problem, oriented toward offline decision making after imagery is captured. Task 2 is a rule based, geometric perception problem using classical HSV thresholding within a continuous real time control loop, translating perception into a motor command roughly ten times per second.

That difference comes from what each task demands, not from any inconsistency in approach. A drone survey can tolerate a per-image delay of a fraction of a second, whereas real-time navigation must produce a control decision on every frame with minimal latency, favouring cheap colour thresholding over a neural network pass for this task. Put side by side, the two tasks make a simple point: robotic vision isn't one technique applied everywhere. It's a set of complementary approaches, each chosen for its computational budget, latency and available training data.

## **Concluding Remarks**

This report has presented two robotic vision systems for the module Advanced Robotics Automation and Computer Vision. Task 1 produced a MobileNetV2 based plant disease classifier trained on PlantVillage, reaching the accuracy and F1 performance reported in the Executive Summary while remaining light enough for embedded deployment. Task 2 produced a ROS 2 perception and navigation design for a simulated TurtleBot3 using HSV colour thresholding and a rule-based controller, verified offline across seven scenarios and confirmed on the live ROSject, matching the controller's proportional gain to five decimal places.

The limitations were stated plainly rather than downplayed: Task 1's reliance on augmentation as a proxy for genuine aerial imagery, and Task 2's dependence on manually tuned colour thresholds. Addressing both, with genuine UAV training data and fuller in-simulation testing, is a clear direction for future work.

## Acknowledgement

Perplexity AI (2026) was used to support literature searching, identify potentially relevant academic sources, and assist with grammar correction and word-limit reduction during editing.

*All sources included in the reference list were checked by the author and cited as the original academic works rather than citing Perplexity AI as a source. The analysis, interpretation, discussion, and final written content submitted in this report are the author's own work and responsibility.*

The architecture and pipeline diagrams in Sections 3.3 and 4.2 were generated using Python scripts with the matplotlib library, written specifically to render the structures described in this report. The scripts and their outputs were reviewed by the author before inclusion.

## References

Jlassi, A., Elaoud, A., Ghazouani, H., and Barhoumi, W. (2024) 'Potato Leaf Disease Classification Using Transfer Learning and Reweighting-Based Training with Imbalanced Data.' SN Computer Science 5, 987. [online] Available from <https://doi.org/10.1007/s42979-024-03334-x> [Accessed 16 September 2026].

Kumar, A., Monga, H.P., Brahma, T., Kalra, S., and Sherif, N. (2025) 'Mobile-Friendly Deep Learning for Plant Disease Detection: A Lightweight CNN Benchmark Across 101 Classes of 33 Crops.' [online] Available from <https://arxiv.org/pdf/2508.10817> [Accessed 4 September 2026].

Manoj, H.M., Shanthi, D.L., Lakshmi, B.N., Archana, K.J., Jyothi, E.V.N., and Archana, K. (2025) 'AI-driven drone technology and computer vision for early detection of crop disease in large agricultural areas.' Scientific Reports 16, 2479. [online] Available from <https://www.nature.com/articles/s41598-025-32384-1> [Accessed 4 September 2026].

mohanty/PlantVillage (2016) [online] Available from <https://huggingface.co/datasets/mohanty/PlantVillage> [Accessed 4 September 2026].

Mohanty, S.P., Hughes, D.P., and Salathe, M. (2016) 'Using Deep Learning for Image-Based Plant Disease Detection.' Frontiers in Plant Science 7, 1419.

Pal, C., Karmakar, S., Mukherjee, I., and Chakrabarti, P.P. (2025) 'A lightweight and explainable CNN model for empowering plant disease diagnosis.' Scientific Reports 15, 30720. [online] Available from <https://www.nature.com/articles/s41598-025-94083-1> [Accessed 4 September 2026].

Pal, O.K., Shovon, M.S.H., Mridha, M.F., and Shin, J. (2023) 'A Comprehensive Review of AI-enabled Unmanned Aerial Vehicle: Trends, Vision, and Challenges.' [online] Available from <https://arxiv.org/pdf/2310.16360> [Accessed 4 September 2026].

Perplexity AI (2026) Response to queries on plant disease classification literature and report editing. Perplexity AI response to Amaan Mohammed Khalander. [online] Available from <https://www.perplexity.ai> [Accessed 16 September 2026].

PlantVillage-Dataset (2016) [online] Available from <https://github.com/spMohanty/PlantVillage-Dataset> [Accessed 4 September 2026].

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., and Ng, A.Y. (2009) 'ROS: an open-source Robot Operating System.' In: ICRA Workshop on Open Source Software. Kobe: IEEE.

ROBOTIS (2024) TurtleBot3 e-Manual [online] Available from <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> [Accessed 4 September 2026].

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L. (2018) 'MobileNetV2: Inverted Residuals and Linear Bottlenecks.' In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Salt Lake City: IEEE, pp.4510–4520.

TheConstruct.ai (2024) TurtleBot3 Courses [online] Available from <https://www.theconstruct.ai> [Accessed 4 September 2026].